2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS) | 978-1-6654-5519-0/22/$31.00 ©2022 IEEE | DOI: 10.1109/FOCS54457.2022.00097

A Hash Table Without Hash Functions, and How to Get the Most Out of Your Random Bits

William Kuszmaul
MIT

***Abstract*—This paper considers the basic question of how strong of a probabilistic guarantee can a hash table, storing n $(1+\Theta(1))\log n$-bit key/value pairs, offer? Past work on this question has been bottlenecked by limitations of the known families of hash functions: The only hash tables to achieve failure probabilities less than $1/2^{\text{polylog}\,n}$ require access to fully-random hash functions—if the same hash tables are implemented using the known explicit families of hash functions, their failure probabilities become $1/\text{poly}(n)$.**

To get around these obstacles, we show how to construct a randomized data structure that has the same guarantees as a hash table, but that *avoids the direct use of hash functions*. Building on this, we able to construct a hash table using $O(n)$ random bits that achieves failure probability $1/n^{n^{1-\varepsilon}}$ for an arbitrary positive constant ε.

In fact, we show that this guarantee can even be achieved by a *succinct dictionary*, that is, by a dictionary that uses space within a $1+o(1)$ factor of the information-theoretic optimum.

Finally we also construct a succinct hash table whose probabilistic guarantees fall on a different extreme, offering a failure probability of $1/\text{poly}(n)$ while using only $\tilde{O}(\log n)$ random bits. This latter result replicates a guarantee previously achieved by Dietzfelbinger et al., but with increased space efficiency and with several surprising technical components.

I. Introduction

A ***dictionary*** is any data structure that supports insertions, deletions, and queries on a set S of up to n ***keys***; dictionaries often also allow for a user to store a value associated with each key, which can then be retrieved during queries. Unless stated otherwise, we will assume a machine word of $\Theta(\log n)$ bits, which means that keys/values are also $O(\log n)$ bits. We will also require implicitly that a dictionary should take at most linear space (i.e., $O(n\log n)$ bits) and that a dictionary should be explicit (i.e., it can be initialized in time $O(n)$). In fact, the dictionaries in this paper have the stronger property that they can be initialized in constant time.

Randomized dictionaries are often also referred to as ***hash tables***.[1] A hash table is said to have ***failure probability*** ε if each operation takes constant time with probability at least $1-\varepsilon$, and is said to ***succeed with high probability*** if $\varepsilon \le 1/\text{poly}\,n$.

A central open question is whether there exists a deterministic constant-time dictionary. A remarkable success in this direction is Pătraşcu and Thorup's dynamic fusion node [1], which builds on older work by Fredman and Willard [2] in order to construct a deterministic constant-time dictionary for very small sets of keys—that is, sets S of at most $\text{polylog}\,n$ keys that are $\Theta(\log n)$ bits each. For sets of $\Theta(n)$ keys, it is widely believed that (even non-explicit) deterministic constant-time dictionaries are impossible [3], but we are still very far from having lower bounds to establish this (see [4]–[8] for other related work on this question).

In this paper, we consider a natural relaxation of this question: What is the smallest failure probability that a hash table can offer [9]–[11]? We present the first hash table to achieve a significantly sub-polynomial failure probability. And we show that such a hash table can even be made ***succinct***, meaning that it uses space within a $(1+o(1))$ factor of the information-theoretic optimum.

Past work on super-high-probability guarantees. The study of probabilistic guarantees for hash tables has, up until now, been intimately tied to the study of hash-function families [12]–[23]. If one has access to fully-random hash functions, then it is known [9]–[11] how to achieve substantially sub-polynomial failure probabilities. However, as observed by Goodrich, Hirschberg, Mitzenmacher, and Thaler [11], the known techniques for simulating constant-time hash functions with high independence [14], [17], [22] are *themselves* randomized constructions that introduce an additional $1/\text{poly}(n)$ probability of failure. Efforts at reducing these failure

[1] Hash tables are sometimes also informally defined as any solution to the dictionary problem that makes use of hash functions. We intentionally take a more open-ended perspective as to the definition of a hash table, so that we include data structures that accomplish the same goal as traditionally accomplished by hash tables, but using different means.

978-1-6654-5519-0/22/$31.00 ©2022 IEEE
DOI 10.1109/FOCS54457.2022.00097

probabilities [11] have only been able to do so at the cost of $\omega(1)$ evaluation times.

Of the known families of constant-time hash functions, the only one that has been successfully used to obtain a hash table with sub-polynomial failure probabilities is tabulation hashing [21]. Although the standard analyses of tabulation hashing include a $1/\text{poly}(n)$ failure probability, it has been noted by [21] that, in some parameter regimes, the true failure probability is actually sub-polynomial. Indeed, one can extend the techniques of [21] to show that tabulation hash functions are load-balancing with probability $1-1/2^{\text{polylog}\,n}$, thereby allowing one to construct a hash table that has a failure probability of $1/2^{\text{polylog}\,n}$. To the best of our knowledge, this remains the smallest failure probability to be achieved by any hash table.

This paper: hash tables with nearly optimal failure probabilities. We introduce a simple data structure, which we call the ***amplified rotated trie***, that offers a failure probability of $1/n^{n^{1-\varepsilon}}$ for an arbitrarily small positive constant ε of our choice. Barring a deterministic constant-time dictionary, this is the close to the strongest guarantee that one could hope for: if there were to exist a hash table with failure probability $1/n^{\varepsilon n}$, for some positive constant $\varepsilon > 0$, then that would imply the existence of a (non-explicit) deterministic constant-time dictionary. Our result improves significantly over the previous state-of-the-art of $1/2^{\text{polylog}\,n}$.

Our second result is that, with a few small modifications, the same data structure can be used to obtain a very different guarantee. The resulting hash table, which we call the ***budget rotated trie***, uses $\tilde{O}(\log n)$ random bits to support constant-time operations with high probability in n. This guarantee, which has also been achieved using more classical hashing-based techniques in previous work by Dietzfelbinger et al. [24], serves as a natural dual to the one above — rather than trying to minimize failure probability, while using up to $O(n)$ random bits, one tries to minimize random bits while maintaining a standard $1/\text{poly}(n)$ failure probability.

An interesting feature of budget rotated tries is that they are able to make use of so called "gradually-increasing-independence hash functions" [19], [20]. These hash functions, introduced originally by Celis, Reingold, Segev, and Wieder [19] (and subsequently made more efficient by Meka, Reingold, Rothblum, and Rothblum [20]) can be used to distribute n balls roughly evenly across n bins using only $O(n \log\log n)$ random bits, but come with the seemingly significant drawback that they require $\Theta((\log\log n)^2)$ time to evaluate. As a consequence, past work on applying these hash functions to classical hash tables [25] has incurred $\omega(1)$ time per operation. Our approach suggests that such gradually-increasing-independence may be more broadly applicable to than was previously thought, and can be used in the design of constant-time data structures.

Achieving succinctness. Finally, we turn our attention to space efficiency. There has also been a great deal of work on how to construct a ***succinct hash table*** (see, e.g., [9], [26]–[28]), that is, a hash table that stores n key/values pairs from a universe U in space

$$(1+o(1))\mathcal{B}(|U|,n)$$

bits, where $\mathcal{B}(|U|,n) = \log\binom{|U|}{n}$ is the information-theoretic lower bound on the size of any hash table.

In the extended version of the paper [29], we show that the data structures in this paper can also be made succinct, in the parameter regime where keys/values are $(1+\Theta(1))\log n$ bits. More generally, we give a black-box transformation that can be applied to any dictionary in order to obtain a succinct dictionary whose probabilistic guarantees are nearly the same as the original's. The new dictionary uses $\mathcal{B}(|U|,n) + O(n(\log n)/\log\log n)$ bits.

Interestingly, the transformation itself makes use of our (non-succinct) budget rotated trie as a critical algorithmic component. The transformation also makes use of a reduction due to Raman and Rao [26], and can be seen as a constant-time and randomness-efficient version of the succinct dictionary given in [26] (which guaranteed only constant expected-time operations).

Applying our transformation, we obtain two data structures: we get a succinct hash table that uses $O(\log n(\log\log n)^3) = \tilde{O}(\log n)$ random bits, while supporting constant-time operations with high probability; and a succinct hash table with a failure probability of $1/n^{n^{1-\varepsilon}}$, where ε is an arbitrarily small positive constant of our choice.

Circumventing the hash-function bottleneck. At the core of our results is a simple but powerful observation: that it is possible to construct a hash table *that does not use hash functions*, and that is consequently free of the limitations that hamper known hash-function constructions. In particular, we begin our exposition by constructing a simple randomized dictionary that we call a ***rotated radix trie***. Like standard hash tables, the rotated radix trie uses linear space and is constant-time (with high probability). But unlike standard hash tables, which rely on randomness supplied by hash functions, the rotated radix trie uses randomness directly embedded into the data structure. The rotated radix trie then serves as the basis for both the amplified rotated trie and the budget rotated trie.

Outline. The rest of the paper proceeds as follows. Section II presents basic preliminaries and conventions. Section III presents and analyzes the rotated radix trie. Building on this, Section IV gives a hash table that achieves failure probability $1/n^{n^{1-\varepsilon}}$ and Section V gives a high-probability hash table using $O(\log n \log\log n)$ random bits; in the extended version of the paper [29], we show that the latter guarantee can also be extended to the case where machine words are $\omega(\log n)$ bits. Finally, in the extended version of the paper [29], we further show how to transform any linear-space hash table into a succinct hash table, while nearly preserving the randomization guarantees of the data structure.

II. Preliminaries and Conventions

We now present several preliminary definitions and conventions for discussing (non-succinct) dictionaries.

Keys, values, and dictionaries. Let $U = [\text{poly}(n)]$ be the set of all possible $\Theta(\log n)$-bit keys, and let $V = [\text{poly}(n)]$ be the set of all possible $\Theta(\log n)$-bit values. A ***dictionary*** is a data structure that stores a set of keys from U, and that associates each key x with a value $y \in V$. Dictionaries support three operations: Insert(x,y) adds key x to the set, if it is not already there, and sets the corresponding value to y; Delete(x) removes x; and Query(x) reports whether key x is present, returning the corresponding value if so.

When discussing non-succinct dictionaries, we focuses (without loss of generality) on fixed-capacity dictionaries, that is, dictionaries that are permitted to have up to n keys at a time. Such dictionaries can be used to implement dynamically-resized dictionaries by simply rebuilding the dictionary (in a deamortized fashion) whenever its size changes by a constant factor. Unless stated otherwise, we shall require implicitly that dictionaries must use at most linear space (i.e., $O(n \log n)$ bits) and have $O(1)$ initialization time.

Standard techniques for simplifying dictionaries. There are several standard reductions that can be used to simplify the problem of maintaining a linear-space dictionary.

We can assume without loss of generality that the lifespan of a dictionary is only $O(n)$ operations. Indeed, longer sequences of operations can be broken into phases of size $O(n)$, and the dictionary can be rebuilt from scratch during each phase (i.e., all of the elements are gradually moved from one instance of the dictionary to another new instance of the dictionary). The rebuild cost can be spread across the phase so that the asymptotic running times of operations are preserved.[2]

[2]For our purposes, rebuilds *do not* sample new random bits. Once a dictionary's random bits are chosen, they are fixed forever.

Since the lifespan of each phase is only $O(n)$ operations, we can implement deletions with the following trivial approach: simply mark elements as deleted, and defer the actual removal of those elements until the next rebuild. As a consequence, when designing the dictionary that will be used to implement each phase, we can assume without loss of generality that the only operations performed are insertions/queries.

We will therefore assume throughout the paper that, whenever we are discussing a non-succinct dictionary, the sequence of operations being performed has length $O(n)$ and consists exclusively of inserts/queries.

Randomization. Randomized dictionaries are given access to a stream of random bits—the dictionary can access the next $\Theta(\log n)$ bits of the stream in time $O(1)$. When analyzing a randomized dictionary, the goal is to bound the ***failure probability*** for any given operation. We emphasize that, in this context, failure does not refer to lack of correctness, but instead to lack of timeliness. A dictionary ***fails*** whenever an operation takes super-constant time.

All of our dictionaries share the property that, once a failure occurs, all of the rest of the operations (in the current phase of $O(n)$ inserts/queries) also fail. We will not bother to explicitly specify the dictionary's behavior when a failure occurs, since at that point it is okay for each of the remaining operations in the phase to take linear time.

We remark that randomized data structures are analyzed against *oblivious* adversaries, meaning that the sequence of insertions/deletions/queries being performed is determined independently of the random bits that the dictionary uses. We also remark that the failure probability of a dictionary is determined on a per-operation basis. For example, if a dictionary has failure-probability p and is used for $1/p$ operations, then it is reasonable that some failures should occur.[3]

III. A Warmup Data Structure: The Rotated Trie.

In this section, we present a simple randomized constant-time dictionary, called the ***rotated radix trie***, that serves as the basis for the data structures in later sections.

The starting place: an n-ary radix trie. The starting place for our data structure will be the classic n-ary radix trie. Each internal node of the trie can be viewed as an array of size n, where the j-th entry of the array stores

[3]Moreover, failures may be correlated between steps (and between phases). For example, if we are using r random bits, and an adversary guesses them, then they can force failures all the time with probability $1/2^r$.

either a pointer to child j, if such a child exists, or a null character otherwise. The leaves of the trie correspond to the keys in the data structure (and are where we store values). In general, there is a leaf with root-to-leaf path $j_1, j_2, j_3, \ldots, j_d$ if and only if the key $j_1 \circ j_2 \circ j_3 \circ \cdots \circ j_d \in [n^d] = [U]$ is present.

What makes the n-ary radix trie an interesting starting place is that the trie deterministically supports constant-time operations. What it does not support is space efficiency: there may be as many as $\Theta(n)$ internal nodes, each of which is an array of size n, and which collectively require space $\Omega(n^2)$ to implement.

Using randomness to save space: the rotated radix trie. We now add randomness to our data structure in a very simple way. Label the internal nodes of the trie by $1, 2, \ldots, m$ for some $m \in O(n)$, and refer to the array used to implement each internal node $i \in [m]$ as A_i. When the data structure is initialized, we assign to each internal node i a ***random rotation*** r_i selected uniformly at random from $\{0, 1, \ldots, n-1\}$. The rotation r_i is stored as part of the node i.

The purpose of r_i is to apply a random cyclic rotation to the array A_i. That is, if a pointer would have been stored in position j of A_i, it is now stored in position $((j + r_i) \bmod n)$ of A_i instead.

Finally, having rotated each of the arrays A_i by r_i, we now overlay the arrays $A_1, A_2, \ldots, A_m$ on top of one another, and we store the contents of all of them in a single array A of size n. Of course, the j-th position of A may be responsible for storing elements from multiple A_is. As long as the number of elements stored in each entry is relatively small, then this is fine: we simply implement each entry of A as a dynamic fusion node [1], which is a deterministic constant-time linear-space dictionary capable of storing up to $\ell = \operatorname{polylog} n$ key/value pairs at a time.

If, prior to collapsing the arrays into a single array A, the the j-th position of rotated array A_i stored a pointer to array $A_{i'}$, then afterwards the dynamic fusion node $A[j]$ stores the key-value pair $(i, (i', r_{i'}))$. In this setting, we refer to the pair $(i', r_{i'})$ as a ***pointer*** to $A_{i'}$, since it dictates which array $A_{i'}$ we are pointing at and where to find the entries of $A_{i'}$. Similarly, if prior to collapsing the arrays, the j-th position of the rotated array A_i stored a pointer p directly to a value (rather than to another array $A_{i'}$), then the dynamic fusion node $A[j]$ stores the key-value pair (i, p).

Analyzing the rotated radix trie. To analyze the rotated radix trie, we must show that, with high probability in n, each entry of A is responsible for storing entries from at most $\ell = \operatorname{polylog} n$ different A_is.

Let us first establish some conventions that will be useful throughout the rest of the paper. When discussing a radix trie, we will refer to the arrays $A_1, A_2, \ldots, A_m$ as the ***nodes*** (or sometimes as the ***internal nodes***), and we will refer to the non-null entries of each A_i (i.e., the entries containing pointers) as ***balls***.

In total, there are $O(n)$ balls in the trie. Each ball b is specified by a pair $(s, c) \in [m] \times [n]$, where $s \in [m]$ is the ***source node*** for the ball (i.e., the node containing the ball), and $c \in [n]$ is the ***child index*** of the ball (i.e., the index in A_i where b is logically stored). The effect of randomly rotating the arrays A_i and then overlaying them to obtain a single array A is that each ball $b = (s, c)$ gets mapped to position $\phi(s, c) := c + r_s$ in A. We refer to the entries of A as ***bins***, so each ball b gets mapped to bin $\phi(b)$. The dynamic fusion node for a each bin $j \in [n]$ stores the set of key-value pairs (b, p) where b ranges over the balls satisfying $\phi(b) = j$, and p is the pointer corresponding to the ball b.

For $i \in [m]$ and $j \in [n]$, let $X_{i,j}$ be the 0-1 random variable indicating whether node i places a ball into bin j. The $X_{i,j}$s are not independent across the bins j, but they are independent across the nodes i, since each $X_{i,j}$ is a function of the random bits r_i. Therefore, the number Y_j of balls in bin j, which is given by $Y_j = \sum_{i=1}^{m} X_{i,j}$, is a sum of independent indicator random variables.

Each of the $O(n)$ balls has probability $1/n$ of being in bin j, so $\mathbb{E}[Y_j] = O(1)$. Thus, by a Chernoff bound, we have that $Y_j \leq \operatorname{polylog} n$ with high probability in n. The Chernoff bound actually tells us that $Y_j \leq \operatorname{polylog} n$ with probability $1/n^{\operatorname{polylog} n}$, so we have even achieved a slightly sub-polynomial probability of failure.

Putting the pieces together. If we implement deletions as in Section II, then we obtain the following result:

Proposition 1. *The rotated radix trie is a randomized linear-space dictionary that can store up to n $\Theta(\log n)$-bit keys/values at a time, and that supports each operation in constant time with probability $1 - 1/n^{\operatorname{polylog} n}$.*

It's worth taking a moment to remark on how to initialize our data structure. The random rotations r_i can be initialized lazily, so that r_i is generated the first time that the node i is used. Additionally, we do not have to actually pay the cost of initializing any arrays, since we can use standard techniques to simulate zero-initialized arrays in constant time (see [30] or Problem 9 of Section 1.6 of [31]). Thus our rotated radix trie can be initialized in constant time.

Taking stock of our situation. The rotated radix trie does not, on its own, make any significant progress on either of the problems that we care about: (1) achieving super-high probability guarantees, and (2) using a near-

logarithmic number of random bits. We have achieved a *slightly* sub-polynomial failure probability, but we are nowhere near our goal of $1/n^{n^{1-\varepsilon}}$.

What makes the rotated radix trie useful, however, is that the role of randomness in the data structure is remarkably simple. The *only* sources of randomness are the rotational offsets $r_1, r_2, \ldots, r_m$. In this sense, the rotated radix trie deviates from the standard mold for how to design a constant-time dictionary. The randomness in the data structure isn't used to hash elements, but is instead used to apply random rotations to sparse arrays.

Since the role of randomness will be important in later sections, we conclude the current section by discussing an important subtlety in how the randomness is repurposed over time. Consider how the data structure evolves over a large period of time containing many insertions/deletions. As the shape of the trie changes, each array A_i will be repurposed to represent different parts of the trie. This means that the way in which the random rotation r_i interacts with the key space also changes over time, with the same r_i applying to a different node in the trie (and thus a different part of the key space) at different times. The re-purposing of r_is has an interesting consequence: even if two points in time t_1 and t_2 store the exact same set S of key/value pairs as one-another, the shape of the rotated trie may differ considerably between the two times.

IV. The Amplified Rotated Trie

In this section, we modify the rotated radix trie to reduce its probability of failure (i.e., the probability that a given operation takes super-constant time) to $1/n^{n^{1-\varepsilon}}$, for a positive constant ε of our choice. We will refer to this new data structure as the ***amplified rotated radix trie***.

Storing overflow balls in a (non-rotated) trie. Whenever a ball b is inserted into a bin j that already contains $\ell = \text{polylog}\, n$ other balls, the ball b is regarded as an ***overflow ball***. Since each bin is a dynamic fusion node with capacity ℓ, we cannot store the overflow balls in the bins.

We instead store the overflow balls in a secondary data structure Q that is implemented as a n^δ-ary trie, for some positive constant $\delta > 0$.

The secondary data structure Q supports inserts/queries on overflow balls in constant time. On the other hand, Q is not space efficient. If there are q overflow balls, then Q may use as much as qn^δ space. To establish that our dictionary uses linear space, we must show that

$$\Pr[q \geq n^{1-\delta}] \leq O\left(1/n^{n^{1-\varepsilon}}\right). \tag{1}$$

The problem: dependencies between balls with shared source nodes. Our current data structure does not yet satisfy (1), however. This is because, whenever multiple balls share the same source node, their assignments become closely linked. Suppose, for example, that the rotated trie R has only 2ℓ internal nodes, and that each internal node $i \in \{1,2,\ldots,2\ell\}$ contains $\Theta(n/\ell)$ balls $(i,1),(i,2),\ldots,(i,\Theta(n/\ell))$. With probability $1/n^{2\ell} = 1/2^{\text{polylog}\, n}$, each internal node $i \in \{1,2,\ldots,2\ell\}$ has random rotation $r_i = 0$. This results in bins $1,2,\ldots,\Theta(n/\ell)$ each containing 2ℓ balls—in other words, half of the balls in the system are overflow balls. This means that

$$\Pr[q \geq \Omega(n)] \geq 1/2^{\text{polylog}\, n}.$$

That is, our failure probability using Q the store overflow balls is *no better* than the failure probability that we achieved in Section III without Q.

Reducing the dependencies. What makes the above pathological example possible is that it is possible to have only a small number of internal nodes in our rotated trie. This makes it so that there are only a small number of random bits that affect the rotated trie's structure, preventing us from achieving any super-high probability guarantees.

To fix this problem, we reduce the fanout of our rotated radix trie from n to n^δ. Now each internal node can contain at most n^δ balls, so there are guaranteed to be at least $n^{1-\delta}$ internal nodes. This ensures that there are always at least $n^{1-\delta}\log n$ random bits affecting the trie's structure.

We remark that, since the fanout of the rotated trie is now n^δ, each ball is determined by a pair (s,c) where $s \in [m]$ is a source node and $c \in [n^\delta]$ is a child index. Nonetheless, the mapping ϕ from balls to bins works exactly as before: we map ball (s,c) to bin $\phi(s,c) = ((r_s + c) \bmod n)$ where $r_s \in [n]$ is selected at random.

Bounding the number of overflow balls. Of course, there are still dependencies between the number q_j of overflow balls in different bins $j \in [n]$. To handle these dependencies, we make use of a tool from probabilistic combinatorics.

Call a function $f : [0,1)^m \to \mathbb{R}$ ***L-Lipschitz*** if for every pair of inputs of the form $\vec{x} = (x_1,\ldots,x_i,\ldots,x_m)$ and $\vec{x'} = (x_1,\ldots x_i',\ldots,x_m)$, we have $|f(\vec{x}) - f(\vec{x'})| \leq L$. McDiarmid's inequality [32] tells us that if f is L-Lipschitz and $X_1, X_2, \ldots, X_m \in [0,1)$ are independent random variables, then for any $t \geq 0$,

$$\Pr[|f(X_1,\ldots,X_m) - \mathbb{E}[f(X_1,\ldots,X_m)]| \geq t] \leq 2e^{-2t^2/(mL^2)}.$$

To apply McDiarmid's inequality to our situation,

define $f(r_1,\ldots,r_m) := q$ to be the number of overflow balls. Observe that f is n^δ-Lipschitz, since each r_i can determine the outcome of at most n^δ different balls. Since $\mathbb{E}[q] = \frac{1}{\text{poly}\, n}$, it follows by McDiarmid's inequality that

$$\begin{aligned}\Pr[f(r_1,\ldots,r_m) \geq n^{1-\delta}] &\leq e^{-\Omega(n^{2-2\delta}/(mn^{2\delta}))} \\ &= e^{-\Omega(n^{2-2\delta}/n^{1+2\delta})} \\ &= e^{-\Omega(n^{1-4\delta})}.\end{aligned}$$

For any $0 < \varepsilon \leq 1$, we can set $\delta = \varepsilon/5$ so that

$$\begin{aligned}\Pr[q \geq n^{1-\delta}] &\leq e^{-\Omega(n^{1-4\delta})} \\ &\leq O\left(n^{-n^{1-\varepsilon}}\right).\end{aligned}$$

This establishes (1). If we implement deletions as in Section II, then we arrive at the following theorem.

Theorem 2. *The $n^{\varepsilon/5}$-ary amplified rotated radix trie is a randomized linear-space dictionary that can store up to n $\Theta(\log n)$-bit keys/values at a time, and that supports each operation in constant time with probability $1 - O\left(1/n^{n^{1-\varepsilon}}\right)$.*

We remark that there is a strong sense in which the amplified rotated radix trie is nearly optimal. In particular, for any constant $\varepsilon > 0$, if there were to exist a randomized linear-space dictionary with failure probability of $1/n^{\varepsilon n}$, that would imply the existence of a deterministic (though non-explicit) linear-space constant-time dictionary.

Lemma 3. *Let $\varepsilon > 0$ be any positive constant and assume a machine word of size $w = \Theta(\log n)$ bits. Suppose there exists randomized linear-space dictionary that stores up to n $\Theta(\log n)$-bit keys/values at a time and has failure probability $1/n^{\varepsilon n}$. Then there also exists a deterministic (not-necessarily explicit) dictionary with the same guarantees.*

Proof. To distinguish the randomized dictionary from the deterministic dictionary that we are constructing, we will refer to the former as a hash table and the latter as a dictionary.

As noted in Section II, by rebuilding our dictionary once every $O(n)$ operations, we can assume without loss of generality that the lifespan of the dictionary is at most $O(n)$ operations. We will implement the dictionary using a hash table with capacity $n' = cn$ for some large positive constant c to be determined later. This means that the hash table has failure probability

$$1/n^{\varepsilon n'} = 1/n^{\varepsilon cn}.$$

Each operation takes place on a $\Theta(\log n)$-bit key/value pair, so there are at most $n^{O(1)}$ options for what a given operation could be. The total number of $O(n)$-long operation sequences is therefore at most $n^{O(n)}$. Since our hash table has failure probability $1/n^{\varepsilon cn}$, its total failure probability on any given sequence of $O(n)$ operations is at most $O(n)/n^{\varepsilon cn} \leq 1/n^{\varepsilon cn/2}$ The probability that there exists *any* sequence of operations on which our hash table fails to be constant-time is therefore at most

$$\frac{n^{O(1)}}{n^{\varepsilon cn/2}},$$

which if c is taken to be a sufficiently large constant, is at most $1/2$. Thus there exists some choice of random bits for which our hash table is constant-time on *every* sequence of operations. By hard-coding in this choice of random bits, we arrive at a deterministic constant-time dictionary.

Note that, since the hash table spends total time $O(n)$ on the $O(n)$ operations, the number of random bits that it can use is at most $O(nw) = O(n \log n)$ bits—thus the deterministic dictionary can hard-code the random bits in linear space. □

Although one typically assumes a machine-word size of $\Theta(\log n)$ bits, it is also an interesting question what the strongest achievable probabilistic guarantees are in the setting where machine words (as well as keys/values) are of some size $w = \omega(\log n)$ bits. On one hand, the larger key size makes it so that Lemma 3 no longer applies, so in principle, one might be able to achieve a failure probability of $1/n^{\omega(n)}$. On the other hand, from an upper-bound perspective, it is not even known how to achieve a *sub-polynomial failure probability* in this setting [9]–[11], [21]. Here, the main obstacle appears to be unavoidably about hash functions: can one construct a family of hash functions from $[2^w]$ to $[\text{poly}(n)]$ such that for any given n-element set $S \subseteq [2^w]$, we have that $\max_{x \in S} |\{y \in S \mid h(x) = h(y)\}| \leq \text{polylog}\, n$ with probability $1/n^{\omega(1)}$? If such a family were to exist, then it could be directly combined with Theorem 2 to construct a dictionary that achieves sub-polynomial failure probability for any key-size w. We conjecture that no such family of hash functions exists, and moreover, that a sub-polynomial failure probability is not possible for word sizes $w = \omega(\log n)$ bits.

V. The Budget Rotated Trie

In this section, we present a dictionary that uses only $O(\log n \log\log n)$ random bits, while guaranteeing that each operation takes constant time with probability $1 - 1/\text{poly}(n)$ (i.e., with high probability in n). We will refer to the data structure as the ***budget rotated trie***. In Appendix A, we further extend the budget rotated trie to support keys that are $\omega(\log n)$ bits, while still using only $O(\log n \log\log n)$ bits of randomness.

We remark that the guarantee achieved by the budget rotated trie is not novel—in fact, a previous approach by Dietzfelbinger, Gil, Matias, and Pippenger [24] can be used to achieve $O(\log n)$ random bits for the setting of $\Theta(\log n)$-bit keys that we are considering. Nonetheless, we believe that the construction for the budget rotated tries is interesting in its own right, both because of its relationship to the amplified rotated trie, and also because of the surprising way in which it is able to make use of gradually-increasing-independence hash functions. Additionally, the specific structure of the budget rotated trie will prove useful in our quest for succinctness in the extended version of the paper [29].

Our starting place is again the rotated trie, and as in Section IV, we will take the fanout of the trie to be n^δ for some constant δ; in fact, it will suffice to simply use $\delta = 1/4$.

Reducing the number of random bits to $O(n/\operatorname{polylog} n)$. To transform the n^δ-ary rotated trie into a budget rotated trie, our first modification will be to reduce the number of random bits from $O(n \log n)$ to $O(n/\operatorname{polylog} n)$. Of course, this may not seem like much progress, but we shall see later that the distinction is important.

Recall that, in a rotated trie, each ball b (i.e., each non-null entry in an internal node) contains a pointer to either a leaf (i.e., an actual key/value pair) or another internal node (i.e., a child). We now add a third option: if the ball should be pointing at another internal node x, but if the subtree rooted at x contains fewer than $\ell = \operatorname{polylog} n$ total keys, then we store that subtree as a dynamic fusion node z. If the size of the subtree rooted at x subsequently surpasses ℓ, then we create an actual internal node for x—in this case, any elements stored in the fusion node z remain in z, and the ball b now stores two pointers, one to x and one to z. In other words, there are now three possible states for a ball: it can contain a pointer to a leaf; it can contain a pointer to a dynamic fusion node; or it can contain two pointers, one to a dynamic fusion node and one to another internal node of the trie.

The point of this modification is that we only create an internal node x if the subtree rooted at x contains at least $\ell = \operatorname{polylog} n$ elements. Importantly, this means that the total number of internal nodes m is at most $O(n/\ell) = n/\operatorname{polylog} n$. The number of random bits needed for the rotations $r_1, r_2, \ldots, r_m$ is therefore also $n/\operatorname{polylog} n$.

Changing the balls-to-bins mapping. Our next modification is to change how we map the balls to bins. Recall that each ball b is specified by a pair (s,c), where $s \in [m]$ is the source node of the ball and $c \in [n^\delta]$ is the child index. In the standard rotated trie, we map balls to bins using the function

$$\phi(s,c) = (c + r_s) \bmod n.$$

We will now instead map balls to bins using the function

$$\psi(s,c) = (c + a_s (\bmod n^\delta)) \cdot n^{1-\delta} + b_s,$$

where a_s is selected at random from $[n^\delta]$ and b_s is selected at random from $[n^{1-\delta}]$.

When can think about ψ as follows. We break the bins into groups $G_1, \ldots, G_{n^\delta}$ of size $n^{1-\delta}$, and we use the random value $a_s \in [n^\delta]$ to assign the ball to a random group. Once the ball is assigned to a group G_i, it is then assigned to the b_s-th bin in that group. Importantly, the assignments are designed so that each source node s assigns *at most one* of its balls to any given group G_i. There will never be two balls b_1, b_2 in group G_i that both obtain their assignments b_s from the same source node.

Since the number m of internal nodes may be as large as $n/\operatorname{polylog} n$, we cannot afford to generate $a_1, a_2, \ldots, a_m \in [n^\delta]$ and $b_1, b_2, \ldots, b_m \in [n^{1-\delta}]$ truly at random. Fortunately, as we shall now see, the roles of the a_is and b_is have been carefully designed so that both sequences can be generated using a small number of "seed" random bits.

Generating the a_is with $O(1)$-independent hash functions. Let k be a sufficiently large positive constant, and select a random hash function $g : [n] \to [n^\delta]$ from a family of k-independent hash functions. Since $k = O(1)$, the function g can be specified using $O(\log n)$ random bits, and can be evaluated in time $O(1)$. We compute the a_is by

$$a_i := g(i).$$

To analyze the number of balls in each group G_i, we use a well-known tail bound for k-independent random variables (see, e.g., [33] or [34]).

Lemma 4 (Lemma 2.2 of [33]). *Let $k \geq 4$ be an even integer. Suppose $X_1, \ldots, X_m$ are k-wise independent 0-1 random variables. Let $X = \sum_i X_i$. Then, for any $t \geq 0$,*

$$\Pr[|X - \mathbb{E}[X]| \geq t] \leq 2 \left(\frac{nk}{t^2} \right)^{k/2}.$$

Define X_j to be the event that source-node j sends a ball to group G_i. The X_js are k-independent, so we have by Lemma 4 that

$$\Pr[|G_i| - \mathbb{E}[|G_i|] \geq n^{0.75}] \leq 2 \left(\frac{kn}{n^{1.5}} \right)^{k/2} \leq n^{-\Omega(k)} = 1/\operatorname{poly}(n).$$

Since $\delta = 0.25$, it follows that

$$\Pr[|G_i| - \mathbb{E}[|G_i|] \geq n^{1-\delta}] \leq 1/\operatorname{poly}(n).$$

Since each of the $O(n)$ balls is equally likely to be in any group, we have that $\mathbb{E}[|G_i|] = O(n^{1-\delta})$. Thus

$$\Pr[|G_i| \leq O(n^{1-\delta})] \geq 1 - 1/\operatorname{poly}(n).$$

That is, each group G_i contains at most $O(n^{1-\delta})$ balls with high probability in n.

Generating the b_is with increasing-independence hash functions. To generate the b_is without using a large number of random bits, we make use of a more sophisticated family of hash functions. Call a family $\mathcal{H}(t)$ of hash functions $h : [\operatorname{poly}(t)] \to [t]$ ***load-balancing*** if it can be used to map t balls to t bins with maximum load $\operatorname{polylog} t$; that is, for any fixed set $S \subseteq \operatorname{poly}(t)$ of size t, and for any fixed $i \in [t]$, if we select a random $h \in \mathcal{H}$, then

$$|\{s \in S \mid h(s) = i\}| \leq \operatorname{polylog} t$$

with probability $1 - 1/\operatorname{poly}(t)$.

Celis, Reingold, Segev, and Wieder [19] showed how to construct a load-balancing family $\mathcal{H}(t)$ of hash functions such that each $h \in \mathcal{H}$ can be described with $O(\log t \log\log t)$ random bits and can be evaluated in time $O(\log t \log\log t)$. The family $\mathcal{H}$ is referred to as having "gradually-increasing-indepenence" because each $h \in \mathcal{H}$ is actually the composition of $\Theta(\log\log t)$ hash functions $h_1, \ldots, h_{\Theta(\log\log t)}$ with different levels of independence: each h_i determines $\Theta((3/4)^i \log t)$ bits of h, and each h_i is $(1/\operatorname{poly} t)$-close to being $\Theta((4/3)^i)$-independent.

The family $\mathcal{H}$ comes with a tradeoff. It is able to achieve a maximum load of $\operatorname{polylog} t$ (in fact, it even achieves maximum load $O(\log t / \log\log t)$) using on $O(\log t \log\log t)$ bits, but it requires super-constant time to evaluate. Subsequent work [20] has improved the evaluation time from $O(\log t \log\log t)$ to $O((\log\log t)^2)$. It seems unlikely that the evaluation time can be improved to $O(1)$, however, since $\Omega(\log\log t)$ time is needed just to read the random bits used to evaluate the hash function.

The super-constant evaluation time makes it so that hash functions with gradually-increasing independence are not suitable for direct use in constant-time hash tables [25]. We get around this problem by using h not as a hash function but as a pseudo-random number generator. Specifically, we select a random $h : [m] \to [n^{1-\delta}]$ from $\mathcal{H}(n^{1-\delta})$, and we use h to initialize the b_is as

$$b_i := h(i).$$

Since h takes time $O((\log\log n)^2)$ to evaluate, each b_i now takes time $O((\log\log n)^2)$ to initialize. Recall, however, that we only create a new internal node x in our rotated trie once there are more than $\ell = \operatorname{polylog} n$ records that want to reside in that node's subtree; the first $\ell = \operatorname{polylog} n$ insertions that wish to use x are instead placed into a dynamic fusion node that acts as a proxy for x. As a result, we can afford to spend up to ℓ time *initializing* the node x, and we can spread that time across the ℓ insertions that trigger x's initialization. Since $\ell = \omega((\log\log n)^2)$, we can initialize $b_i = h(i)$ without any problem.

Analyzing the maximum load. Recall that, with probability $1 - 1/\operatorname{poly}(n)$, each group G_i contains at most $O(n^{1-\delta})$ balls. Furthermore, each of the balls have different source nodes than one another. If a ball has source-node s, then it is placed in the b_s-th bin of G_i.

Let $S_i \subseteq [m]$ be the set of source nodes that assign balls to G_i. Then for each $r \in [n^{1-\delta}]$, the number $g_{i,r}$ of balls in the r-th bin of G_i is given by

$$g_{i,r} = |\{s \in S_i \mid h(s) = r\}|.$$

Since $h : \operatorname{poly}(n) \to [n^{1-\delta}]$ is from a load-balancing family of hash functions, we are guaranteed to have

$$g_{i,r} \leq \operatorname{polylog} n^{1-\delta} \leq \operatorname{polylog} n$$

with high probability in n.

Putting the pieces together. The fact that each bin contains at most $\operatorname{polylog} n$ balls (with high probability) means that, as in the standard rotated trie, each bin can be implemented with a dynamic fusion node. Operations on our dictionary therefore take time $O(1)$ with high probability in n. If we implement deletions as in Section II, then we arrive at the following theorem.

Theorem 5. *The budget rotated trie is a randomized linear-space dictionary that can store up to n $\Theta(\log n)$-bit keys/values at a time, that uses $O(\log n \log\log n)$ random bits, and that supports each operation in constant time with probability $1 - 1/\operatorname{poly}(n)$.*

We conclude the section by observing that there is a strong sense in which the guarantee achieved by the budget rotated trie is optimal. In particular, if there were to exist a hash table failure probability $1/n^c$ but that used fewer than $c \log n$ random bits, then there would also necessarily exist a deterministic linear-space constant-time dictionary.

Lemma 6. *Suppose there exists a randomized linear-space dictionary that can store up to n $\Theta(\log n)$-bit keys/values at a time, that uses $c \log n$ random bits, but that has a failure probability smaller than $1/n^c$. Then there exists a deterministic dictionary with the same guarantees.*

Proof. To distinguish the randomized dictionary from the deterministic dictionary that we are constructing, we will refer to the former as a hash table. Let R denote the $c \log n$ random bits used by the hash table. Define $\mathcal{D}$ to be the deterministic dictionary obtained by setting

$R = 0$. Suppose for contradiction that D is not constant time. Then there exists some sequence of operations such that the final operation on D takes super-constant time. This means that, with probability at least $1/n^c$, that same operation would have taken super-constant time in our hash table. But the hash table has failure probability smaller than $1/n^c$, a contradiction. □

APPENDIX

In this section, we extend the budget rotated trie to support keys from a universe U of super-polynomial size. Throughout the section, we set $U = [2^u]$ for some $u = n^{o(1)}$, and we assume that machine words are $\Theta(u)$ bits.

To support large keys, the natural approach is to first hash elements from U to a smaller universe U' of polynomial size, an then to store the $\Theta(\log n)$-bit keys in a hash table along with pointers to the full keys/values. Past work on load-balancing hash functions [19] has used a pair-wise independent hash function $h : [2^u] \to [\text{poly}(n)]$ to perform this reduction. This requires the use of $\Theta(u)$ random bits, which when u is large, is significantly larger than $\log n \log\log n$.

An appealing alternative to using pairwise-independent hash functions would be to instead use Pagh's construction [35] (which, in turn, is based on an earlier construction by Fredman, Komlós, and Semerédi [36]) of $(1 + o(1))$-universal hash functions that require only $O(\log n + \log\log u)$ random bits. The only minor problem with this construction is that it is not fully explicit. The construction requires access to a random prime number $p \in [\text{poly}(n)]$, but the only known time-efficient high-probability approaches to constructing such a prime number require $\omega(\log n \log\log n)$ random bits (see discussion in [37]).

Fortunately, this issue is relatively straightforward to solve. For completeness, we now give a construction for a simple family of hash functions that can be initialized in time $o(n)$ and used for universe reduction.

Lemma 7. *Let $n > u^c$ for a sufficiently large positive constant c and let $S \subseteq [2^u]$ be a set of size n. Let $\mathcal{P}$ be the set of prime numbers in the range $[n^{2/c}]$. Select $p_1, p_2, \ldots, p_{c^2}$ independently and uniformly at random from $\mathcal{P}$, and define the function $h : [2^u] \to [n^{2c}]$ by*

$$h(x) = (x \bmod p_1 p_2 \cdots p_{c^2}).$$

With probability $1 - 1/\text{poly}(n)$, h is injective on S.

Proof. The probability $\Pr[|h(S)| \neq S]$ is at most

$$\sum_{s_1, s_2 \in S} \Pr[|s_1 - s_2| \text{ divisible by all of } p_1, p_2, \ldots, p_{c^2}],$$

where s_1 and s_2 are implicitly taken to be distinct. Since the p_is are independent, this is

$$\sum_{s_1, s_2 \in S} (\Pr[|s_1 - s_2| \text{ divisible by } p_1])^{c^2}.$$

The quantity $|s_1 - s_2|$ is an element of $U = [2^u]$, and can thus have at most u distinct prime factors. Therefore,

$$\begin{aligned}\Pr[|h(S)| \neq S] &\leq \sum_{s_1, s_2 \in S} \left(\frac{u}{|\mathcal{P}|}\right)^{c^2} \\ &\leq \sum_{s_1, s_2 \in S} \left(\frac{n^{1/c}}{|\mathcal{P}|}\right)^{c^2}.\end{aligned}$$

By the Prime Number Theorem, the set $\mathcal{P}$ of primes in the range $[n^{2/c}]$ has size $\Omega(n^{2/c}/\log n)$. Therefore,

$$\begin{aligned}\Pr[|h(S)| \neq S] &\leq \sum_{s_1, s_2 \in S} O\left(\frac{n^{1/c}}{n^{2/c}/\log n}\right)^{c^2} \\ &\leq O\left(\sum_{s_1, s_2 \in S} \left(\frac{\log n}{n^{1/c}}\right)^{c^2}\right) \\ &\leq O\left(\sum_{s_1, s_2 \in S} \frac{\log^{c^2} n}{n^c}\right) \\ &\leq O\left(\frac{n^2 \log^{c^2} n}{n^c}\right) \\ &\leq 1/\text{poly}(n).\end{aligned}$$

□

Since all of the prime numbers in $[n^\varepsilon]$ can be enumerated in time $O(n^{2\varepsilon})$, we get the following corollary:

Corollary 8. *Let $u = n^{o(1)}$. For any constant $\delta > 0$, there exists an explicit family $\mathcal{H}$ of constant-time hash functions $h : [2^u] \to [\text{poly}(n)]$ such that (a) a random function $h \in \mathcal{H}$ can be constructed in time $O(n^\delta)$ using $O(\log n)$ random bits; and (b) for any fixed set $S \subseteq U$ of size n, and for a random $h \in \mathcal{H}$, we have that $|h(U)| = |U|$ with probability $1 - 1/\text{poly}(n)$.*

We can use Corollary 8 to construct a version of the budget rotated trie that supports large universes.

Theorem 9. *Let $u = n^{o(1)}$, suppose that keys/values are u bits, and assume a machine word of size at least $\Omega(u)$ bits. The budget rotated trie uses $O(\log n \log\log n)$ random bits, it uses $O(nu)$ bits of space, and it supports insert/delete/query operations on up to n keys/values at a time. The data structure can be initialized in time $O(n^\varepsilon)$, for a positive constant ε of our choice, and each insert/delete/query operation takes constant time with probability $1 - 1/\text{poly}(n)$.*

To eliminate the $O(n^\varepsilon)$ initialization cost, we can also construct a dynamic version of the same data structure, where there is some upper bound N on the data structure's size, but where the true size n changes over time. Every time that the data structure's size changes by a constant factor, we rebuild it based on the new value of n. Each rebuild takes time $O(n)$ (with high probability in n), but the cost of a rebuild can be spread across $\Theta(n)$ operations. The properties of this new data structure can be summarized with the following corollary.

Corollary 10. *Let $u = N^{o(1)}$, suppose that keys/values are u bits, and assume a machine word of size at least $\Omega(u)$ bits. The dynamic budget rotated trie uses $O(\log N \log\log N)$ random bits and supports insert/delete/query operations on up to N keys/values at a time. If it is storing n key/value pairs, then it uses $O(nu)$ bits of space, and each insert/delete/query operation takes constant time with probability $1 - 1/\operatorname{poly}(n)$.*

ACKNOWLEDGEMENTS

The author is grateful to Martin Dietzfelbinger for helpful pointers to prior work, and would also like to thank Michael A. Bender and Martín Farach-Colton for a number of insightful technical discussions.

This research was partially sponsored by the United States Air Force Research Laboratory and the United States Air Force Artificial Intelligence Accelerator and was accomplished under Cooperative Agreement Number FA8750-19-2-1000. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the United States Air Force or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

The research was also partially supported by a Hertz Fellowship and an NSF GRFP Fellowship.

REFERENCES

[1] M. Patrascu and M. Thorup, "Dynamic integer sets with optimal rank, select, and predecessor search," in *2014 IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS)*, 2014, pp. 166–175.

[2] M. L. Fredman and D. E. Willard, "Blasting through the information theoretic barrier with fusion trees," in *Proceedings of the twenty-second annual ACM Symposium on Theory of Computing (STOC)*, 1990, pp. 1–7.

[3] M. Thorup, "Mihai Pătraşcu: Obituary and open problems," *Bulletin of EATCS*, vol. 1, no. 109, 2013.

[4] R. Sundar, "A lower bound for the dictionary problem under a hashing model," in *Proceedings 32nd Annual Symposium of Foundations of Computer Science (FOCS)*, 1991, pp. 612–621. [Online]. Available: https://doi.org/10.1109/SFCS.1991.185427

[5] T. Hagerup, P. B. Miltersen, and R. Pagh, "Deterministic dictionaries," *Journal of Algorithms*, vol. 41, no. 1, pp. 69–85, 2001. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S019667740191171X

[6] M. Ružić, "Uniform deterministic dictionaries," *ACM Trans. Algorithms*, vol. 4, no. 1, Mar. 2008. [Online]. Available: https://doi.org/10.1145/1328911.1328912

[7] R. Pagh, "Faster deterministic dictionaries," in *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, USA, 2000, pp. 487–493.

[8] M. Dietzfelbinger, A. Karlin, K. Mehlhorn, F. auf der Heide, H. Rohnert, and R. Tarjan, "Dynamic perfect hashing: upper and lower bounds," in *Proceedings of the 29th Annual Symposium on Foundations of Computer Science (FOCS)*, 1988, pp. 524–531.

[9] M. A. Bender, A. Conway, M. Farach-Colton, W. Kuszmaul, and G. Tagliavini, "All-purpose hashing," *arXiv preprint arXiv:2109.04548*, 2021.

[10] M. T. Goodrich, D. S. Hirschberg, M. Mitzenmacher, and J. Thaler, "Fully de-amortized cuckoo hashing for cache-oblivious dictionaries and multimaps," *arXiv preprint arXiv:1107.4378*, 2011.

[11] ——, "Cache-oblivious dictionaries and multimaps with negligible failure probability," in *Mediterranean Conference on Algorithms*. Springer, 2012, pp. 203–218.

[12] M. Naor and O. Reingold, "On the construction of pseudorandom permutations: Luby—Rackoff revisited," *Journal of Cryptology*, vol. 12, no. 1, pp. 29–66, 1999.

[13] M. Luby and C. Rackoff, "How to construct pseudorandom permutations from pseudorandom functions," *SIAM Journal on Computing*, vol. 17, no. 2, pp. 373–386, 1988.

[14] A. Pagh and R. Pagh, "Uniform hashing in constant time and optimal space," *SIAM Journal on Computing*, vol. 38, no. 1, pp. 85–96, 2008.

[15] A. Ostlin and R. Pagh, "Uniform hashing in constant time and linear space," in *Proceedings of the thirty-fifth annual ACM symposium on Theory of Computing (STOC)*, 2003, pp. 622–628.

[16] E. Kaplan, M. Naor, and O. Reingold, "Derandomized constructions of k-wise (almost) independent permutations," *Algorithmica*, vol. 55, no. 1, pp. 113–133, 2009.

[17] M. Dietzfelbinger and P. Woelfel, "Almost random graphs with simple hash functions," in *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing (STOC)*, 2003, pp. 629–638.

[18] M. Dietzfelbinger and F. M. auf der Heide, "A new universal class of hash functions and dynamic hashing in real time," in *International Colloquium on Automata, Languages, and Programming (ICALP)*. Springer, 1990, pp. 6–19.

[19] L. E. Celis, O. Reingold, G. Segev, and U. Wieder, "Balls and bins: Smaller hash families and faster evaluation," in *Proceedings of the 2011 IEEE 52nd Annual Symposium on Foundations of Computer Science (FOCS)*, 2011, pp. 599–608.

[20] R. Meka, O. Reingold, G. N. Rothblum, and R. D. Rothblum, "Fast pseudorandomness for independence and load balancing," in *International Colloquium on Automata, Languages, and Programming (ICALP)*. Springer, 2014, pp. 859–870.

[21] M. Pătraşcu and M. Thorup, "The power of simple tabulation hashing," *Journal of the ACM (JACM)*, vol. 59, no. 3, pp. 1–50, 2012.

[22] A. Siegel, "On universal classes of extremely random constant-time hash functions," *SIAM Journal on Computing*, vol. 33, no. 3, pp. 505–543, 2004.

[23] ——, "On universal classes of fast high performance hash functions, their time-space tradeoff, and their applications," in *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*, 1989, pp. 20–25.

[24] M. Dietzfelbinger, J. Gil, Y. Matias, and N. Pippenger, "Polynomial hash functions are reliable," in *International Colloquium on Automata, Languages, and Programming*. Springer, 1992, pp. 235–246.

[25] O. Reingold, R. D. Rothblum, and U. Wieder, "Pseudorandom graphs in data structures," in *International Colloquium on Automata, Languages, and Programming (ICALP)*. Springer, 2014, pp. 943–954.

[26] R. Raman and S. S. Rao, "Succinct dynamic dictionaries and

trees," in *Automata, Languages and Programming*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 357–368.
[27] M. A. Bender, M. Farach-Colton, J. Kuszmaul, W. Kuszmaul, and M. Liu, "On the optimal time/space tradeoff for hash tables," in *Proceedings of the Fifty-Fourth Annual ACM Symposium on Theory of Computing (STOC)*, 2022.
[28] M. Liu, Y. Yin, and H. Yu, "Succinct filters for sets of unknown sizes," in *47th International Colloquium on Automata, Languages, and Programming (ICALP)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.
[29] W. Kuszmaul, "A hash table without hash functions, and how to get the most out of your random bits," *arXiv preprint arxiv:2209.06038*, 2022.
[30] A. V. Aho and J. E. Hopcroft, *The design and analysis of computer algorithms*. Pearson Education India, 1974.
[31] J. Bentley, *Programming pearls*. Addison-Wesley Professional, 2016.
[32] C. McDiarmid, "On the method of bounded differences," *Surveys in combinatorics*, vol. 141, no. 1, pp. 148–188, 1989.
[33] M. Bellare and J. Rompel, "Randomness-efficient oblivious sampling," in *Proceedings 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 276–287.
[34] D. P. Dubhashi and A. Panconesi, *Concentration of measure for the analysis of randomized algorithms*. Cambridge University Press, 2009.
[35] R. Pagh, "Dispersing hash functions," *Random Structures & Algorithms*, vol. 35, no. 1, pp. 70–82, 2009.
[36] M. L. Fredman, J. Komlos, and E. Szemeredi, "Storing a sparse table with o(1) worst case access time," in *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, 1982, pp. 165–169.
[37] P.-A. Fouque and M. Tibouchi, "Close to uniform prime number generation with fewer random bits," in *International Colloquium on Automata, Languages, and Programming (ICALP)*. Springer, 2014, pp. 991–1002.